WHAT IF THE KERNEL LIES?

Hunting Linux rootkits



MEGA SEKURAK HACKING PARTY

sekurak HACKING PARTY

whoami

PROFESSIONAL

- Global Threat Defense Engineering Director at PepsiCo
- Active member of ISSA International
- Ex-Symantec, ex-ISSA Polska Board Member
- CISSP since 2010 till recently
- OSEP too
- Locked Shields veteran

PERSONAL

- Father of two to-be hackers
- Coffee lover: more ways to brew it than php had vulnerabilities
- Homegrown researcher and builder of home appliances
- Snowboarder in winter, biker all year



sekurak HACKING PARTY

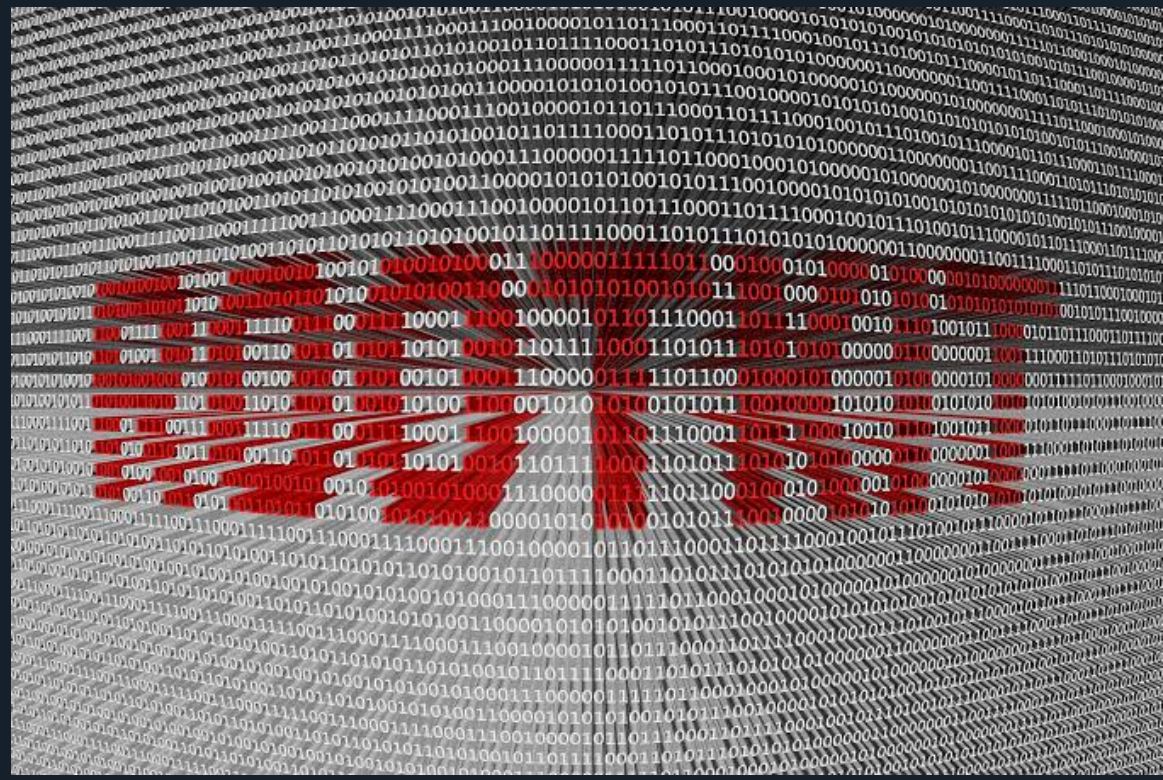
Rootkit: the system lies to you

Software that **hides an attacker's presence** so they keep control of a compromised machine.

Example: a process keeps running but disappears from `ps`. Nothing stopped; you just can't see it.

Hiding ≠ privileges ≠ persistence.

Today: three rootkits on a lab machine. What evidence is left?



sekurak HACKING PARTY

Linux basics: user space and kernel space

Two execution domains, separated by a **privilege boundary**.

User space — applications

Shells, browsers, services, tools such as `ls`, and libraries.

Processes normally have isolated memory and restricted CPU privileges.

Kernel space — the OS core

CPU scheduling, memory, filesystems, networking, drivers.

Privileged code that manages resources and access to hardware.

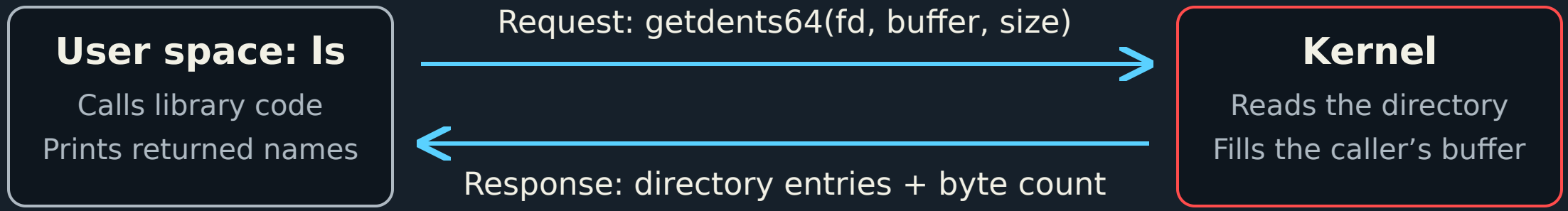
Even a root shell runs in user space.

Programs request kernel services through **system calls**.

Your program asks. The kernel answers.

A **system call** asks the kernel to perform an operation:

read a directory, open a file, or create a process.



The syscall **number** selects the operation; **arguments** supply its inputs.

The kernel returns a **result or error** and may write data into a buffer.

A module brings code into the kernel

A **loadable kernel module (LKM)** extends the running kernel.

Its code executes with **kernel privileges**.

Stage	What happens
Build	C source becomes a `.ko` file: code, data, and metadata
Load	The loader allocates memory, resolves symbols, and calls the init function
Unload	The exit function cleans up; the kernel releases module memory

`struct module` tracks the loaded module and its memory regions.

Drivers and rootkits can enter through the same door.

A hook puts code on the path

A **hook** runs additional code at a chosen point in an operation. It can observe the operation or change its behavior.



Example: read a directory, then remove names from the returned list.

A hook can also **replace the operation entirely**.

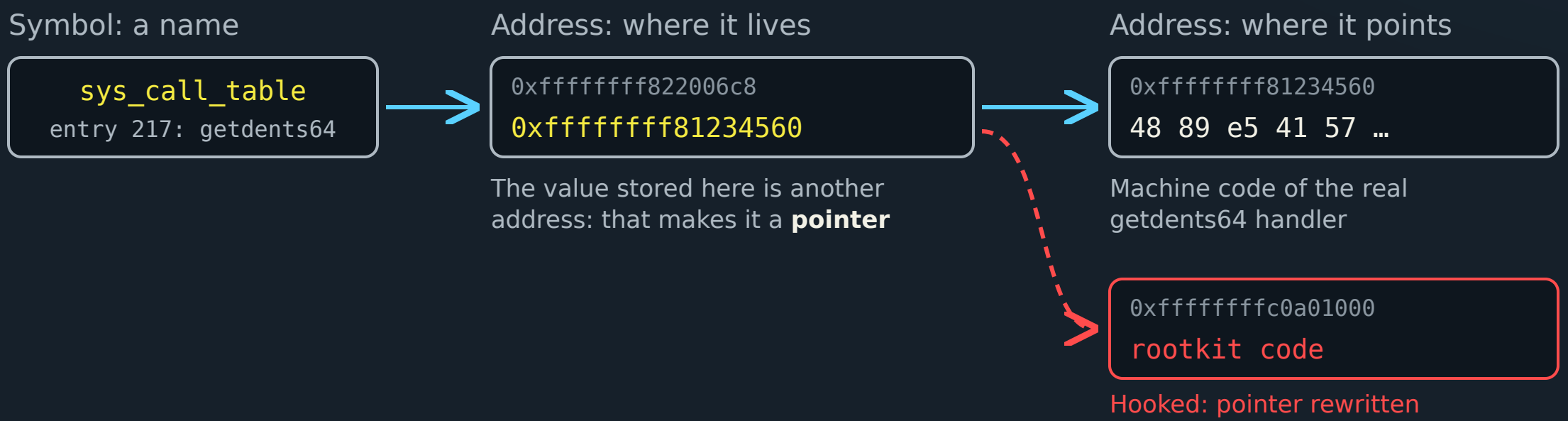
How do you hook execution?

Space	Approach	What changes?	Where it acts
User	`LD_PRELOAD` — library interposition	Library function lookup in the process	Dynamically linked user processes
User	`ptrace` — process control	Traced process's registers and memory	Selected user processes
Kernel	Direct patching — the classic	Syscall-table pointers or kernel instructions	Kernel code / data — direct memory writes
Kernel	ftrace — registered callbacks	Callbacks on instrumented function entry	Kernel functions
Kernel	eBPF — programs loaded from user space	Verified programs attached to kernel hooks	Supported kernel hooks

eBPF: kernel-side behavior without loading your own `.ko`.

Permissions, kernel configuration, and attachment type determine what is possible.

Follow an address: name → location → contents



An **address** is a location. A **symbol** names it. A **pointer** holds one.

A hook rewrites a pointer: same slot, different destination.

vmlinux: the kernel before it booted

File	What's in it
`/boot/vmlinuz-$(uname -r)`	Compressed boot image; `extract-vmlinux` unpacks the ELF
`vmlinux` from the debug package	Code bytes as built, symbol table, DWARF types
`/boot/System.map-$(uname -r)`	Symbol → link-time address; no code
`/sys/kernel/btf/vmlinux`	Types and struct layouts; no addresses

```
$ nm vmlinux | grep -w _stext
ffffffff81000000 T _stext
```

Our known-good baseline: what the code and pointers should be.

Only if it matches this exact build.

kallsyms and kcore: what we see, and why

```
$ sudo grep -w _stext /proc/kallsyms
ffffffffa1000000 T _stext
$ sudo readelf -lW /proc/kcore     # LOAD segments = kernel text,
                                   # modules, direct map, vmalloc
```

	/proc/kallsyms	/proc/kcore
What	Every symbol the kernel knows: core and loaded modules	Live kernel virtual memory as an ELF core file
Why it exists	Backtraces, ftrace, kprobes, BPF	Debugging the running kernel with GDB
Without root	Addresses read as zero (kptr_restrict)	No access

Both are the running kernel describing itself. A rootkit runs there too.

Kernel memory comes with a maths tax

Got an address? Great. It's not a file offset.

```
0xffffffff81234560   ≈ 16 EiB: near the top of the 64-bit address space
ls -l /proc/kcore    → 128T, a sparse file, and nowhere near that big
```

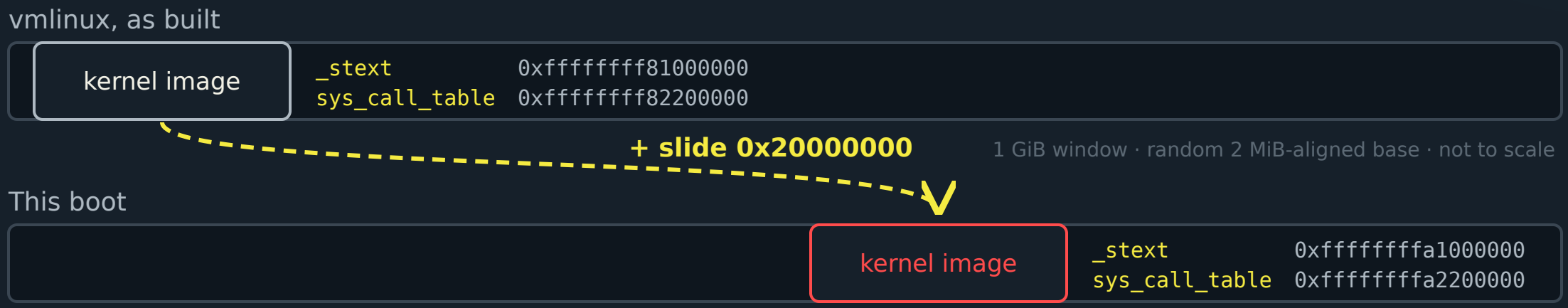
It's a **virtual address**. kcore's ELF segments map address ranges to file offsets:

```
$ sudo readelf -lW /proc/kcore
  LOAD  Offset 0x00007fff81001000  VirtAddr 0xffffffff81000000   ← kernel text

offset = 0x7fff81001000 + (0xffffffff81234560 − 0xffffffff81000000)
       = 0x7fff81235560          → seek there, read the bytes
```

KASLR: the kernel moves every boot

Name → address → bytes works. **Then the kernel moves.**



Why: a random base each boot, so exploits can't hard-code addresses.

Beat it: everything moves together. **One known address gives the slide.**

Do the maths: two images of one kernel

Expected: vmlinux + slide			Live: read through kcore	
slot 0 read	0xffffffffa1456780	=	slot 0 read	0xffffffffa1456780
slot 217 getdents64	0xffffffffa1234560	≠ a lead	slot 217 getdents64	0xffffffffc0a01000
x64_sys_call	e8 1b 4c 23 00 …	=	x64_sys_call	e8 1b 4c 23 00 …

Relocate vmlinux, read kcore: **two images that line up.**

A mismatch is a lead to explain. A match covers only what was compared.

That's why I built lkmhunter



Tools to **detect**, and sometimes **disarm**, running rootkits.

- **Baseline:** matching vmlinux, slide measured and cross-checked
- **Syscalls:** live table and dispatcher vs. the expected image
- **Ftrace:** callback → the module that owns it
- **Disarm:** original bytes written back over a hook

Let's get our hands dirty

Three samples. Three reasons to look again.

Diamorphine
Which syscall paths changed?
Can visibility be restored?

KoviD
Who owns this ftrace callback?
What survives in memory?

Singularity
Is our evidence being filtered before we see it?



Better than detecting: don't let it load

```
module.sig_enforce=1            # or CONFIG_MODULE_SIG_FORCE=y
$ sudo insmod diamorphine.ko
insmod: ERROR: could not insert module diamorphine.ko:
        Key was rejected by service
```

All three samples were **unsigned `.ko` files**: one boot parameter kills that path. Lockdown enforces it too.

Signed ≠ harmless, and eBPF or kernel exploits don't need a `.ko`.

Detection tells you it happened. Prevention means it didn't.

FOUND IT. REMOVED IT.
CLEAN?

No. Don't fight it. Reimage.

"NOTHING FOUND" NEEDS A SECOND SENTENCE:
WHAT DID YOU INSPECT, AND WHAT COULD YOU NOT SEE?

THANK YOU

Questions?

MEGA SEKURAK HACKING PARTY

References

lkmhunter

- github.com/ponquersohn/lkmhunter
- Documentation: CLI, Python API, memory helper

Samples

- Diamorphine: syscall-table and dispatcher hooks
- KoviD: ftrace hooks, hidden module
- Singularity: filtered evidence

Kernel documentation

- syscalls(2): the user/kernel interface
- Ftrace callbacks: hooking through ftrace
- /proc: kallsyms and kcore
- kptr_restrict: hiding kernel pointers
- BTF: types without vmlinux
- Kernel self-protection: KASLR
- Module signing: `sig_enforce`
- Kernel lockdown

BACKUP SLIDES

Unlinking is not unloading

```
Visible list:   A ↔ B ↔ C
After unlink:   A ←———→ C

Memory:         B's code, data, and callbacks remain
```

DKOM: direct kernel object manipulation.

`/proc/modules` and `/sys/module` expose different views.

An object can disappear from one view and survive elsewhere.

Diamorphine changes where the dispatcher jumps

C: select a syscall

```c
// x86-64 Linux 6.9+, abbreviated
long x64_sys_call(const struct pt_regs *regs,
                  unsigned int nr)
{
    switch (nr) {
    case 217:
        return __x64_sys_getdents64(regs);
    // ... other syscall numbers ...
    default:
        return __x64_sys_ni_syscall(regs);
    }
}
```

Assembly: branch to its handler

```asm
; Illustrative compiler output
; ESI = nr, RDI = regs
cmp esi, 217
je  .getdents64
; ... other cases ...

.getdents64:
; before / after Diamorphine:
jmp __x64_sys_getdents64
jmp hacked_getdents64
; Same branch site. New destination.
```

The patch: `E9 [rel32]` — keep the opcode, change the four offset bytes.

Destination = address of the next instruction + signed `rel32`. `E8` calls work the same way.

Undo it: restore the original 5 bytes from matching, relocated `vmlinux`; read back and rescan. In the demo, repair directory hooks first; keep `kill` to unhide, then unload.

Linux: x64_sys_call · Diamorphine: flipswitch_func

KoviD changes where ftrace returns

At the start of a kernel function

```
filldir64:
    ; tracing off: NOP at entry
    ; tracing on:
    call ftrace_trampoline
    ; original function body follows
```

The trampoline saves context, calls registered callbacks, then restores context and resumes execution.

Normal tracing: resume the original body.

KoviD: resume at its replacement instead.

Inside KoviD's callback

```
// fh_ftrace_thunk, abbreviated
regs = ftrace_get_regs(fregs);
hook = container_of(ops,
                    struct ftrace_hook, ops);

if (!within_module(parent_ip, THIS_MODULE))
    regs->ip = (unsigned long)hook->function;
```

`SAVE_REGS | IPMODIFY` allows the saved IP to change. The replacement can hide entries, then call the original; the caller check avoids redirecting that call again.

The same pattern appears in other rootkits: register a callback → redirect → filter results.

Follow the callback: `ftrace_ops.func` → module memory → owner.

Tracing and livepatch also register callbacks; inspect what the callback does.

Kernel: ftrace callbacks and IPMODIFY · KoviD: fh_ftrace_thunk